```
import os
import cv2
import numpy as np
import tifffile as tiff
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch.utils.data import random_split

from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay

from transformers import (
    SegformerForSemanticSegmentation
)


device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)
```

```
Device: cuda
```

```
root_dir =
"/kaggle/input/datasets/rishikeshgopal121/landslide/ReadyToBeUsedDatas
et"

class SegFormerDataset(Dataset):

    def __init__(self, root_dir):

        self.samples = []

        for region in os.listdir(
            os.path.join(root_dir, "6days")
        ):

            region_path = os.path.join(
                root_dir,
                "6days",
                region
            )
```

```
            folders = os.listdir(
                region_path
            )

            for folder in folders:

                if folder == "Segmentations":
                    continue

                coh_path = os.path.join(
                    region_path,
                    folder,
                    "coherence"
                )

                phase_path = os.path.join(
                    region_path,
                    folder,
                    "phase"
                )

                mask_path = os.path.join(
                    region_path,
                    "Segmentations",
                    folder
                )

                if not (
                    os.path.exists(coh_path)
                    and
                    os.path.exists(phase_path)
                    and
                    os.path.exists(mask_path)
                ):
                    continue

                files = sorted(
                    os.listdir(coh_path)
                )

                for file in files:

                    if file.endswith(".tif"):

                        self.samples.append(

                            (

                                os.path.join(
                                    coh_path,
```

```
                                    file
                                ),

                                os.path.join(
                                    phase_path,
                                    file
                                ),

                                os.path.join(
                                    mask_path,
                                    file
                                )

                            )

                        )

        print(
            "Total Samples:",
            len(self.samples)
        )

    def __len__(self):

        return len(
            self.samples
        )

    def __getitem__(self, idx):

        coh_file, phase_file, mask_file = self.samples[idx]

        # ============================
        # LOAD FILES
        # ============================

        coherence = tiff.imread(
            coh_file
        ).astype(np.float32)

        phase = tiff.imread(
            phase_file
        ).astype(np.float32)

        mask = tiff.imread(
            mask_file
        ).astype(np.float32)

        # ============================
        # PHASE DECOMPOSITION
        # ============================
```

```
        cos_phase = np.cos(
            phase
        )

        sin_phase = np.sin(
            phase
        )

        # ============================
        # CREATE INPUT
        # ============================

        image = np.stack(

            [

                coherence,

                cos_phase,

                sin_phase

            ],

            axis=0

        )

        mask = np.expand_dims(

            mask,

            axis=0

        )

        # ============================
        # HORIZONTAL FLIP
        # ============================

        if np.random.rand() > 0.5:

            image = np.flip(
                image,
                axis=2
            ).copy()

            mask = np.flip(
                mask,
                axis=2
```

```
            ).copy()

        # ============================
        # VERTICAL FLIP
        # ============================

        if np.random.rand() > 0.5:

            image = np.flip(
                image,
                axis=1
            ).copy()

            mask = np.flip(
                mask,
                axis=1
            ).copy()

        # ============================
        # ROTATION
        # ============================

        k = np.random.randint(
            0,
            4
        )

        image = np.rot90(

            image,

            k,

            axes=(1, 2)

        ).copy()

        mask = np.rot90(

            mask,

            k,

            axes=(1, 2)

        ).copy()

        # ============================
        # TRANSPOSE
        # ============================
```

```
        if np.random.rand() > 0.5:

            image = np.transpose(

                image,

                (0, 2, 1)

            ).copy()

            mask = np.transpose(

                mask,

                (0, 2, 1)

            ).copy()

        # ==========================
        # Z-SCORE NORMALIZATION
        # ==========================

        image = image.astype(
            np.float32
        )

        for c in range(
            image.shape[0]
        ):

            mean = image[c].mean()

            std = image[c].std()

            image[c] = (

                image[c] - mean

            ) / (

                std + 1e-8

            )

        mask = mask.astype(
            np.float32
        )

        return (

            torch.tensor(
```

```
                image,
                dtype=torch.float32
            ),

            torch.tensor(
                mask,
                dtype=torch.float32
            )

        )

dataset = SegFormerDataset(
    root_dir
)
```

```
Total Samples: 4655
```

```
train_size = int(0.70 * len(dataset))

val_size = int(0.15 * len(dataset))

test_size = len(dataset) - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(

    dataset,

    [
        train_size,
        val_size,
        test_size
    ]
)

print("Train:", len(train_dataset))
print("Validation:", len(val_dataset))
print("Test:", len(test_dataset))
```

```
Train: 3258
Validation: 698
Test: 699
```

```
train_loader = DataLoader(

    train_dataset,

    batch_size=8,

    shuffle=True
)
```

```
test_loader = DataLoader(
    test_dataset,
    batch_size=8,
    shuffle=False
)
val_loader = DataLoader(
    val_dataset,
    batch_size=8,
    shuffle=False
)
class SegFormerModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.segformer =
SegformerForSemanticSegmentation.from_pretrained(
            "nvidia/segformer-b0-finetuned-ade-512-512",
            num_labels=1,
            ignore_mismatched_sizes=True
        )
    def forward(self, x):
        outputs = self.segformer(
            pixel_values=x
        )
        logits = outputs.logits
        logits = F.interpolate(
            logits,
            size=(100, 100),
            mode="bilinear",
```

```
            align_corners=False

        )

        return logits

model = SegFormerModel().to(device)

print(model)
```

{"model_id":"79d8663210944f95b3bd2e909097138f","version_major":2,"version_minor":0}

```
SegformerForSemanticSegmentation LOAD REPORT from: nvidia/segformer-b0-finetuned-ade-512-512
Key                            | Status   |

-------------------------------+----------
+-------------------------------------------------------------------------------
-------------------------------
decode_head.classifier.weight | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150, 256, 1, 1]) vs model:torch.Size([1, 256, 1, 1])
decode_head.classifier.bias   | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([150]) vs model:torch.Size([1])

Notes:
- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

SegFormerModel(
  (segformer): SegformerForSemanticSegmentation(
    (segformer): SegformerModel(
      (encoder): SegformerEncoder(
        (patch_embeddings): ModuleList(
          (0): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(3, 32, kernel_size=(7, 7), stride=(4, 4), padding=(3, 3))
            (layer_norm): LayerNorm((32,), eps=1e-05, elementwise_affine=True)
          )
          (1): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
            (layer_norm): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
          )
          (2): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(64, 160, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))
            (layer_norm): LayerNorm((160,), eps=1e-05,
```

```
elementwise_affine=True)
          )
          (3): SegformerOverlapPatchEmbeddings(
            (proj): Conv2d(160, 256, kernel_size=(3, 3), stride=(2,
2), padding=(1, 1))
            (layer_norm): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
          )
        )
        (block): ModuleList(
          (0): ModuleList(
            (0): SegformerLayer(
              (layer_norm_1): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=32, out_features=32,
bias=True)
                  (key): Linear(in_features=32, out_features=32,
bias=True)
                  (value): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(32, 32, kernel_size=(8, 8), stride=(8,
8))
                  (layer_norm): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): Identity()
              (layer_norm_2): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=32, out_features=128,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(128, 128, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=128)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=128, out_features=32,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
```

```
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=32, out_features=32,
bias=True)
                  (key): Linear(in_features=32, out_features=32,
bias=True)
                  (value): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(32, 32, kernel_size=(8, 8), stride=(8,
8))
                  (layer_norm): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=32, out_features=32,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.014285714365541935)
              (layer_norm_2): LayerNorm((32,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=32, out_features=128,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(128, 128, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=128)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=128, out_features=32,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
          (1): ModuleList(
            (0): SegformerLayer(
              (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=64, out_features=64,
bias=True)
```

```
                  (key): Linear(in_features=64, out_features=64,
bias=True)
                  (value): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(64, 64, kernel_size=(4, 4), stride=(4,
4))
                  (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.02857142873108387)
              (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=64, out_features=256,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=256, out_features=64,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=64, out_features=64,
bias=True)
                  (key): Linear(in_features=64, out_features=64,
bias=True)
                  (value): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(64, 64, kernel_size=(4, 4), stride=(4,
4))
                  (layer_norm): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
                )
```

```
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=64, out_features=64,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.04285714402794838)
              (layer_norm_2): LayerNorm((64,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=64, out_features=256,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(256, 256, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=256)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=256, out_features=64,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
          (2): ModuleList(
            (0): SegformerLayer(
              (layer_norm_1): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=160, out_features=160,
bias=True)
                  (key): Linear(in_features=160, out_features=160,
bias=True)
                  (value): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(160, 160, kernel_size=(2, 2),
stride=(2, 2))
                  (layer_norm): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.05714285746216774)
              (layer_norm_2): LayerNorm((160,), eps=1e-05,
```

```
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=160, out_features=640,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(640, 640, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=640)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=640, out_features=160,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=160, out_features=160,
bias=True)
                  (key): Linear(in_features=160, out_features=160,
bias=True)
                  (value): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                  (sr): Conv2d(160, 160, kernel_size=(2, 2),
stride=(2, 2))
                  (layer_norm): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=160, out_features=160,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.0714285746216774)
              (layer_norm_2): LayerNorm((160,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=160, out_features=640,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(640, 640, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=640)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=640, out_features=160,
```

```
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
          (3): ModuleList(
            (0): SegformerLayer(
              (layer_norm_1): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=256, out_features=256,
bias=True)
                  (key): Linear(in_features=256, out_features=256,
bias=True)
                  (value): Linear(in_features=256, out_features=256,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=256, out_features=256,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.08571428805589676)
              (layer_norm_2): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=256, out_features=1024,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(1024, 1024, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1024)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=1024, out_features=256,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
            (1): SegformerLayer(
              (layer_norm_1): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (attention): SegformerAttention(
                (self): SegformerEfficientSelfAttention(
                  (query): Linear(in_features=256, out_features=256,
bias=True)
                  (key): Linear(in_features=256, out_features=256,
```

```
bias=True)
                  (value): Linear(in_features=256, out_features=256,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
                (output): SegformerSelfOutput(
                  (dense): Linear(in_features=256, out_features=256,
bias=True)
                  (dropout): Dropout(p=0.0, inplace=False)
                )
              )
              (drop_path): SegformerDropPath(p=0.10000000149011612)
              (layer_norm_2): LayerNorm((256,), eps=1e-05,
elementwise_affine=True)
              (mlp): SegformerMixFFN(
                (dense1): Linear(in_features=256, out_features=1024,
bias=True)
                (dwconv): SegformerDWConv(
                  (dwconv): Conv2d(1024, 1024, kernel_size=(3, 3),
stride=(1, 1), padding=(1, 1), groups=1024)
                )
                (intermediate_act_fn): GELUActivation()
                (dense2): Linear(in_features=1024, out_features=256,
bias=True)
                (dropout): Dropout(p=0.0, inplace=False)
              )
            )
          )
        )
        (layer_norm): ModuleList(
          (0): LayerNorm((32,), eps=1e-05, elementwise_affine=True)
          (1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
          (2): LayerNorm((160,), eps=1e-05, elementwise_affine=True)
          (3): LayerNorm((256,), eps=1e-05, elementwise_affine=True)
        )
      )
    )
    (decode_head): SegformerDecodeHead(
      (linear_c): ModuleList(
        (0): SegformerMLP(
          (proj): Linear(in_features=32, out_features=256, bias=True)
        )
        (1): SegformerMLP(
          (proj): Linear(in_features=64, out_features=256, bias=True)
        )
        (2): SegformerMLP(
          (proj): Linear(in_features=160, out_features=256, bias=True)
        )
        (3): SegformerMLP(
```

```
          (proj): Linear(in_features=256, out_features=256, bias=True)
        )
      )
      (linear_fuse): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1,
1), bias=False)
      (batch_norm): BatchNorm2d(256, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
      (activation): ReLU()
      (dropout): Dropout(p=0.1, inplace=False)
      (classifier): Conv2d(256, 1, kernel_size=(1, 1), stride=(1, 1))
    )
  )
)

sample_batch = next(iter(train_loader))

images, masks = sample_batch

images = images.to(device)

with torch.no_grad():

    outputs = model(images)

print("Input Shape :", images.shape)

print("Output Shape:", outputs.shape)
```

```
Input Shape : torch.Size([8, 3, 100, 100])
Output Shape: torch.Size([8, 1, 100, 100])
```

```
# ========================================
# DICE LOSS
# ========================================

def dice_loss(pred, target):

    pred = torch.sigmoid(pred)

    pred = pred.view(-1)

    target = target.view(-1)

    intersection = (

        pred * target

    ).sum()
```

```
    dice = (
        2 * intersection + 1e-8
    ) / (
        pred.sum()
        + target.sum()
        + 1e-8
    )
    return 1 - dice


# ========================================
# FOCAL LOSS
# ========================================
def focal_loss(
    pred,
    target,
    alpha=0.8,
    gamma=2
):
    bce = F.binary_cross_entropy_with_logits(
        pred,
        target,
        reduction="none"
    )
    pt = torch.exp(-bce)
    focal = (
        alpha
        *
        ((1 - pt) ** gamma)
```

```
        *
        bce
    )
    return focal.mean()

# ========================================
# TVERSKY LOSS
# ========================================
def tversky_loss(
    pred,
    target,
    alpha=0.3,
    beta=0.7
):
    pred = torch.sigmoid(pred)
    pred = pred.view(-1)
    target = target.view(-1)
    TP = (
        pred * target
    ).sum()
    FP = (
        pred * (1 - target)
    ).sum()
    FN = (
        (1 - pred) * target
    ).sum()
    tversky = (
```

```
        TP + 1e-8
    ) / (
        TP
        + alpha * FP
        + beta * FN
        + 1e-8
    )
    return 1 - tversky


# ========================================
# HYBRID LOSS
# ========================================
def hybrid_loss(
    pred,
    target
):
    d = dice_loss(
        pred,
        target
    )
    f = focal_loss(
        pred,
        target
    )
    t = tversky_loss(
        pred,
        target
    )
```

```
    total_loss = (
        0.3 * d
        +
        0.2 * f
        +
        0.5 * t
    )
    return total_loss
optimizer = optim.AdamW(
    model.parameters(),
    lr=0.0001,
    weight_decay=1e-4
)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=5
)
train_losses = []
train_acc = []
train_prec = []
train_rec = []
epochs = 120
best_loss = float('inf')
for epoch in range(epochs):
```

```
    model.train()
    total_loss = 0
    correct = 0
    total = 0
    TP = 0
    FP = 0
    FN = 0
    print(f"\nEpoch {epoch+1}/{epochs}")
    for i, (images, masks) in enumerate(train_loader):
        images = images.to(device)
        masks = masks.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = hybrid_loss(
            outputs,
            masks
        )
        loss.backward()
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            1.0
        )
        optimizer.step()
        total_loss += loss.item()
        # ================= METRICS =================
        pred = torch.sigmoid(outputs)
```

```
        pred_bin = (
            pred > 0.5
        ).float()
        correct += (
            pred_bin == masks
        ).sum().item()
        total += masks.numel()
        TP += (
            pred_bin * masks
        ).sum().item()
        FP += (
            pred_bin * (1 - masks)
        ).sum().item()
        FN += (
            (1 - pred_bin) * masks
        ).sum().item()
        progress = (
            (i+1)
            /
            len(train_loader)
        ) * 100
        print(
            f"\rProgress: {progress:.2f}% | Loss: {loss.item():.4f}",
            end=""
        )
    # ================ EPOCH METRICS ================
```

```
    acc = correct / total
    precision = TP / (
        TP + FP + 1e-8
    )
    recall = TP / (
        TP + FN + 1e-8
    )
    train_losses.append(
        total_loss
    )
    train_acc.append(
        acc
    )
    train_prec.append(
        precision
    )
    train_rec.append(
        recall
    )
    print(
        f"\nLoss: {total_loss:.4f}"
    )
    print(
        f"Accuracy : {acc:.4f}"
    )
    print(
```

```
        f"Precision: {precision:.4f}"
    )
    print(
        f"Recall    : {recall:.4f}"
    )
    scheduler.step(
        total_loss
    )
    if total_loss < best_loss:
        best_loss = total_loss
        torch.save(
            model.state_dict(),
            "segformer_best.pth"
        )
        print(
            "Model Saved!"
        )
```

```
Epoch 1/120
Progress: 100.00% | Loss: 0.8001
Loss: 246.7535
Accuracy : 0.9145
Precision: 0.2285
Recall   : 0.6623
Model Saved!

Epoch 2/120
Progress: 100.00% | Loss: 0.1837
Loss: 165.1406
Accuracy : 0.9707
Precision: 0.5491
Recall   : 0.6511
Model Saved!
```

```
Epoch 3/120
Progress: 100.00% | Loss: 0.5747
Loss: 142.9213
Accuracy : 0.9744
Precision: 0.6017
Recall   : 0.6746
Model Saved!

Epoch 4/120
Progress: 100.00% | Loss: 0.1269
Loss: 128.9186
Accuracy : 0.9765
Precision: 0.6306
Recall   : 0.7050
Model Saved!

Epoch 5/120
Progress: 100.00% | Loss: 0.3384
Loss: 122.5235
Accuracy : 0.9782
Precision: 0.6547
Recall   : 0.7245
Model Saved!

Epoch 6/120
Progress: 100.00% | Loss: 0.3647
Loss: 116.7365
Accuracy : 0.9783
Precision: 0.6548
Recall   : 0.7308
Model Saved!

Epoch 7/120
Progress: 100.00% | Loss: 0.3709
Loss: 114.8424
Accuracy : 0.9793
Precision: 0.6707
Recall   : 0.7397
Model Saved!

Epoch 8/120
Progress: 100.00% | Loss: 0.5161
Loss: 111.0478
Accuracy : 0.9801
Precision: 0.6851
Recall   : 0.7392
Model Saved!

Epoch 9/120
```

```
Progress: 100.00% | Loss: 0.8281
Loss: 106.1087
Accuracy : 0.9804
Precision: 0.6849
Recall   : 0.7568
Model Saved!

Epoch 10/120
Progress: 100.00% | Loss: 0.3578
Loss: 107.0085
Accuracy : 0.9810
Precision: 0.6991
Recall   : 0.7513

Epoch 11/120
Progress: 100.00% | Loss: 0.1803
Loss: 100.3928
Accuracy : 0.9818
Precision: 0.7091
Recall   : 0.7661
Model Saved!

Epoch 12/120
Progress: 100.00% | Loss: 0.1475
Loss: 100.0856
Accuracy : 0.9819
Precision: 0.7099
Recall   : 0.7712
Model Saved!

Epoch 13/120
Progress: 100.00% | Loss: 0.8000
Loss: 100.8833
Accuracy : 0.9818
Precision: 0.7060
Recall   : 0.7750

Epoch 14/120
Progress: 100.00% | Loss: 0.8198
Loss: 98.3851
Accuracy : 0.9825
Precision: 0.7184
Recall   : 0.7790
Model Saved!

Epoch 15/120
Progress: 100.00% | Loss: 0.1296
Loss: 96.6526
Accuracy : 0.9830
Precision: 0.7279
```

```
Recall   : 0.7794
Model Saved!

Epoch 16/120
Progress: 100.00% | Loss: 0.2785
Loss: 93.7434
Accuracy : 0.9832
Precision: 0.7274
Recall   : 0.7912
Model Saved!

Epoch 17/120
Progress: 100.00% | Loss: 0.1663
Loss: 89.8463
Accuracy : 0.9836
Precision: 0.7317
Recall   : 0.7978
Model Saved!

Epoch 18/120
Progress: 100.00% | Loss: 0.8001
Loss: 89.9272
Accuracy : 0.9842
Precision: 0.7434
Recall   : 0.7989

Epoch 19/120
Progress: 100.00% | Loss: 0.8107
Loss: 89.8709
Accuracy : 0.9842
Precision: 0.7414
Recall   : 0.8027

Epoch 20/120
Progress: 100.00% | Loss: 0.1227
Loss: 86.5778
Accuracy : 0.9845
Precision: 0.7455
Recall   : 0.8076
Model Saved!

Epoch 21/120
Progress: 100.00% | Loss: 0.3972
Loss: 86.1772
Accuracy : 0.9848
Precision: 0.7504
Recall   : 0.8106
Model Saved!

Epoch 22/120
```

```
Progress: 100.00% | Loss: 0.1284
Loss: 81.6414
Accuracy : 0.9850
Precision: 0.7506
Recall   : 0.8191
Model Saved!

Epoch 23/120
Progress: 100.00% | Loss: 0.3656
Loss: 84.2650
Accuracy : 0.9851
Precision: 0.7544
Recall   : 0.8156

Epoch 24/120
Progress: 100.00% | Loss: 0.1999
Loss: 80.1742
Accuracy : 0.9855
Precision: 0.7632
Recall   : 0.8182
Model Saved!

Epoch 25/120
Progress: 100.00% | Loss: 0.3348
Loss: 79.7673
Accuracy : 0.9857
Precision: 0.7635
Recall   : 0.8267
Model Saved!

Epoch 26/120
Progress: 100.00% | Loss: 0.1362
Loss: 76.2041
Accuracy : 0.9860
Precision: 0.7682
Recall   : 0.8267
Model Saved!

Epoch 27/120
Progress: 100.00% | Loss: 0.3417
Loss: 79.2054
Accuracy : 0.9857
Precision: 0.7626
Recall   : 0.8279

Epoch 28/120
Progress: 100.00% | Loss: 0.3731
Loss: 80.1851
Accuracy : 0.9859
Precision: 0.7651
```

```
Recall   : 0.8289

Epoch 29/120
Progress: 100.00% | Loss: 0.2533
Loss: 76.3439
Accuracy : 0.9865
Precision: 0.7741
Recall   : 0.8372

Epoch 30/120
Progress: 100.00% | Loss: 0.2691
Loss: 74.1655
Accuracy : 0.9867
Precision: 0.7802
Recall   : 0.8353
Model Saved!

Epoch 31/120
Progress: 100.00% | Loss: 0.8012
Loss: 75.7398
Accuracy : 0.9866
Precision: 0.7769
Recall   : 0.8350

Epoch 32/120
Progress: 100.00% | Loss: 0.1834
Loss: 71.5583
Accuracy : 0.9872
Precision: 0.7836
Recall   : 0.8487
Model Saved!

Epoch 33/120
Progress: 100.00% | Loss: 0.8000
Loss: 71.8366
Accuracy : 0.9871
Precision: 0.7816
Recall   : 0.8467

Epoch 34/120
Progress: 100.00% | Loss: 0.0822
Loss: 71.2780
Accuracy : 0.9872
Precision: 0.7846
Recall   : 0.8465
Model Saved!

Epoch 35/120
Progress: 100.00% | Loss: 0.0977
Loss: 69.7709
```

```
Accuracy : 0.9873
Precision: 0.7862
Recall   : 0.8495
Model Saved!

Epoch 36/120
Progress: 100.00% | Loss: 0.2897
Loss: 71.0690
Accuracy : 0.9872
Precision: 0.7850
Recall   : 0.8470

Epoch 37/120
Progress: 100.00% | Loss: 0.0502
Loss: 68.8082
Accuracy : 0.9876
Precision: 0.7902
Recall   : 0.8531
Model Saved!

Epoch 38/120
Progress: 100.00% | Loss: 0.0625
Loss: 67.7308
Accuracy : 0.9876
Precision: 0.7921
Recall   : 0.8507
Model Saved!

Epoch 39/120
Progress: 100.00% | Loss: 0.1087
Loss: 67.0154
Accuracy : 0.9879
Precision: 0.7934
Recall   : 0.8579
Model Saved!

Epoch 40/120
Progress: 100.00% | Loss: 0.1082
Loss: 65.4988
Accuracy : 0.9882
Precision: 0.7990
Recall   : 0.8602
Model Saved!

Epoch 41/120
Progress: 100.00% | Loss: 0.8000
Loss: 65.6547
Accuracy : 0.9882
Precision: 0.7984
Recall   : 0.8620
```

```
Epoch 42/120
Progress: 100.00% | Loss: 0.1187
Loss: 64.9651
Accuracy : 0.9883
Precision: 0.8013
Recall   : 0.8593
Model Saved!

Epoch 43/120
Progress: 100.00% | Loss: 0.7029
Loss: 65.5144
Accuracy : 0.9883
Precision: 0.8006
Recall   : 0.8629

Epoch 44/120
Progress: 100.00% | Loss: 0.1000
Loss: 63.3500
Accuracy : 0.9886
Precision: 0.8057
Recall   : 0.8632
Model Saved!

Epoch 45/120
Progress: 100.00% | Loss: 0.1157
Loss: 64.2478
Accuracy : 0.9887
Precision: 0.8073
Recall   : 0.8659

Epoch 46/120
Progress: 100.00% | Loss: 0.0663
Loss: 63.6998
Accuracy : 0.9885
Precision: 0.8016
Recall   : 0.8693

Epoch 47/120
Progress: 100.00% | Loss: 0.1261
Loss: 63.5994
Accuracy : 0.9887
Precision: 0.8075
Recall   : 0.8652

Epoch 48/120
Progress: 100.00% | Loss: 0.1932
Loss: 62.9715
Accuracy : 0.9889
Precision: 0.8103
```

```
Recall   : 0.8687
Model Saved!

Epoch 49/120
Progress: 100.00% | Loss: 0.5268
Loss: 61.8999
Accuracy : 0.9889
Precision: 0.8079
Recall   : 0.8738
Model Saved!

Epoch 50/120
Progress: 100.00% | Loss: 0.8000
Loss: 62.5446
Accuracy : 0.9889
Precision: 0.8101
Recall   : 0.8692

Epoch 51/120
Progress: 100.00% | Loss: 0.0913
Loss: 59.0240
Accuracy : 0.9893
Precision: 0.8155
Recall   : 0.8770
Model Saved!

Epoch 52/120
Progress: 100.00% | Loss: 0.8000
Loss: 59.3411
Accuracy : 0.9893
Precision: 0.8137
Recall   : 0.8777

Epoch 53/120
Progress: 100.00% | Loss: 0.1331
Loss: 57.9424
Accuracy : 0.9895
Precision: 0.8177
Recall   : 0.8787
Model Saved!

Epoch 54/120
Progress: 100.00% | Loss: 0.2737
Loss: 58.5190
Accuracy : 0.9893
Precision: 0.8139
Recall   : 0.8790

Epoch 55/120
Progress: 100.00% | Loss: 0.1262
```

```
Loss: 57.2983
Accuracy : 0.9896
Precision: 0.8214
Recall   : 0.8787
Model Saved!

Epoch 56/120
Progress: 100.00% | Loss: 0.8000
Loss: 58.4367
Accuracy : 0.9896
Precision: 0.8188
Recall   : 0.8836

Epoch 57/120
Progress: 100.00% | Loss: 0.1533
Loss: 58.3725
Accuracy : 0.9895
Precision: 0.8174
Recall   : 0.8799

Epoch 58/120
Progress: 100.00% | Loss: 0.2036
Loss: 56.4259
Accuracy : 0.9896
Precision: 0.8199
Recall   : 0.8817
Model Saved!

Epoch 59/120
Progress: 100.00% | Loss: 0.1667
Loss: 58.2074
Accuracy : 0.9896
Precision: 0.8182
Recall   : 0.8816

Epoch 60/120
Progress: 100.00% | Loss: 0.8014
Loss: 57.4429
Accuracy : 0.9898
Precision: 0.8227
Recall   : 0.8843

Epoch 61/120
Progress: 100.00% | Loss: 0.1613
Loss: 54.7496
Accuracy : 0.9900
Precision: 0.8266
Recall   : 0.8857
Model Saved!
```

```
Epoch 62/120
Progress: 100.00% | Loss: 0.1087
Loss: 55.3560
Accuracy : 0.9900
Precision: 0.8247
Recall   : 0.8876

Epoch 63/120
Progress: 100.00% | Loss: 0.2106
Loss: 54.8847
Accuracy : 0.9900
Precision: 0.8246
Recall   : 0.8875

Epoch 64/120
Progress: 100.00% | Loss: 0.1174
Loss: 54.3283
Accuracy : 0.9902
Precision: 0.8285
Recall   : 0.8906
Model Saved!

Epoch 65/120
Progress: 100.00% | Loss: 0.2111
Loss: 53.8942
Accuracy : 0.9902
Precision: 0.8266
Recall   : 0.8912
Model Saved!

Epoch 66/120
Progress: 100.00% | Loss: 0.0598
Loss: 55.8005
Accuracy : 0.9902
Precision: 0.8278
Recall   : 0.8884

Epoch 67/120
Progress: 100.00% | Loss: 0.2915
Loss: 52.6669
Accuracy : 0.9902
Precision: 0.8259
Recall   : 0.8924
Model Saved!

Epoch 68/120
Progress: 100.00% | Loss: 0.8070
Loss: 54.1013
Accuracy : 0.9903
Precision: 0.8294
```

```
Recall   : 0.8902

Epoch 69/120
Progress: 100.00% | Loss: 0.1185
Loss: 50.7790
Accuracy : 0.9907
Precision: 0.8348
Recall   : 0.8957
Model Saved!

Epoch 70/120
Progress: 100.00% | Loss: 0.8075
Loss: 52.4367
Accuracy : 0.9907
Precision: 0.8360
Recall   : 0.8944

Epoch 71/120
Progress: 100.00% | Loss: 0.2314
Loss: 51.4391
Accuracy : 0.9906
Precision: 0.8338
Recall   : 0.8947

Epoch 72/120
Progress: 100.00% | Loss: 0.8000
Loss: 51.1005
Accuracy : 0.9908
Precision: 0.8365
Recall   : 0.8967

Epoch 73/120
Progress: 100.00% | Loss: 0.8000
Loss: 51.8744
Accuracy : 0.9907
Precision: 0.8357
Recall   : 0.8955

Epoch 74/120
Progress: 100.00% | Loss: 0.1642
Loss: 50.1137
Accuracy : 0.9908
Precision: 0.8366
Recall   : 0.8985
Model Saved!

Epoch 75/120
Progress: 100.00% | Loss: 0.8000
Loss: 51.1601
Accuracy : 0.9909
```

```
Precision: 0.8361
Recall   : 0.9038

Epoch 76/120
Progress: 100.00% | Loss: 0.0896
Loss: 51.4611
Accuracy : 0.9907
Precision: 0.8356
Recall   : 0.8955

Epoch 77/120
Progress: 100.00% | Loss: 0.8000
Loss: 50.7482
Accuracy : 0.9909
Precision: 0.8384
Recall   : 0.9001

Epoch 78/120
Progress: 100.00% | Loss: 0.0962
Loss: 50.1582
Accuracy : 0.9909
Precision: 0.8399
Recall   : 0.8959

Epoch 79/120
Progress: 100.00% | Loss: 0.2020
Loss: 48.7046
Accuracy : 0.9911
Precision: 0.8414
Recall   : 0.9014
Model Saved!

Epoch 80/120
Progress: 100.00% | Loss: 0.0763
Loss: 48.3690
Accuracy : 0.9910
Precision: 0.8396
Recall   : 0.9024
Model Saved!

Epoch 81/120
Progress: 100.00% | Loss: 0.3229
Loss: 50.5860
Accuracy : 0.9912
Precision: 0.8421
Recall   : 0.9030

Epoch 82/120
Progress: 100.00% | Loss: 0.0456
Loss: 48.0495
```

```
Accuracy : 0.9912
Precision: 0.8406
Recall   : 0.9062
Model Saved!

Epoch 83/120
Progress: 100.00% | Loss: 0.0746
Loss: 47.9341
Accuracy : 0.9911
Precision: 0.8415
Recall   : 0.9011
Model Saved!

Epoch 84/120
Progress: 100.00% | Loss: 0.1187
Loss: 46.6799
Accuracy : 0.9913
Precision: 0.8439
Recall   : 0.9061
Model Saved!

Epoch 85/120
Progress: 100.00% | Loss: 0.1008
Loss: 46.6849
Accuracy : 0.9914
Precision: 0.8457
Recall   : 0.9044

Epoch 86/120
Progress: 100.00% | Loss: 0.2257
Loss: 48.8093
Accuracy : 0.9913
Precision: 0.8448
Recall   : 0.9036

Epoch 87/120
Progress: 100.00% | Loss: 0.1300
Loss: 47.7388
Accuracy : 0.9914
Precision: 0.8447
Recall   : 0.9063

Epoch 88/120
Progress: 100.00% | Loss: 0.3767
Loss: 46.8361
Accuracy : 0.9915
Precision: 0.8472
Recall   : 0.9083

Epoch 89/120
```

```
Progress: 100.00% | Loss: 0.0678
Loss: 45.3526
Accuracy : 0.9916
Precision: 0.8487
Recall   : 0.9082
Model Saved!

Epoch 90/120
Progress: 100.00% | Loss: 0.0545
Loss: 44.7771
Accuracy : 0.9917
Precision: 0.8511
Recall   : 0.9076
Model Saved!

Epoch 91/120
Progress: 100.00% | Loss: 0.2351
Loss: 45.7576
Accuracy : 0.9916
Precision: 0.8501
Recall   : 0.9077

Epoch 92/120
Progress: 100.00% | Loss: 0.0857
Loss: 44.1813
Accuracy : 0.9917
Precision: 0.8513
Recall   : 0.9101
Model Saved!

Epoch 93/120
Progress: 100.00% | Loss: 0.1166
Loss: 44.8184
Accuracy : 0.9917
Precision: 0.8499
Recall   : 0.9105

Epoch 94/120
Progress: 100.00% | Loss: 0.1772
Loss: 45.3394
Accuracy : 0.9916
Precision: 0.8499
Recall   : 0.9083

Epoch 95/120
Progress: 100.00% | Loss: 0.0883
Loss: 45.3440
Accuracy : 0.9918
Precision: 0.8527
Recall   : 0.9109
```

```
Epoch 96/120
Progress: 100.00% | Loss: 0.1574
Loss: 45.6593
Accuracy : 0.9917
Precision: 0.8487
Recall   : 0.9128

Epoch 97/120
Progress: 100.00% | Loss: 0.1142
Loss: 44.5733
Accuracy : 0.9919
Precision: 0.8522
Recall   : 0.9151

Epoch 98/120
Progress: 100.00% | Loss: 0.0788
Loss: 43.5997
Accuracy : 0.9920
Precision: 0.8568
Recall   : 0.9126
Model Saved!

Epoch 99/120
Progress: 100.00% | Loss: 0.8000
Loss: 44.0831
Accuracy : 0.9920
Precision: 0.8552
Recall   : 0.9121

Epoch 100/120
Progress: 100.00% | Loss: 0.1149
Loss: 43.6257
Accuracy : 0.9920
Precision: 0.8558
Recall   : 0.9137

Epoch 101/120
Progress: 100.00% | Loss: 0.0796
Loss: 42.3157
Accuracy : 0.9921
Precision: 0.8559
Recall   : 0.9148
Model Saved!

Epoch 102/120
Progress: 100.00% | Loss: 0.1535
Loss: 42.6212
Accuracy : 0.9920
Precision: 0.8557
```

```
Recall   : 0.9145

Epoch 103/120
Progress: 100.00% | Loss: 0.3814
Loss: 43.2350
Accuracy : 0.9921
Precision: 0.8564
Recall   : 0.9140

Epoch 104/120
Progress: 100.00% | Loss: 0.0969
Loss: 41.7075
Accuracy : 0.9923
Precision: 0.8587
Recall   : 0.9183
Model Saved!

Epoch 105/120
Progress: 100.00% | Loss: 0.1065
Loss: 42.3854
Accuracy : 0.9921
Precision: 0.8568
Recall   : 0.9162

Epoch 106/120
Progress: 100.00% | Loss: 0.0461
Loss: 42.3628
Accuracy : 0.9921
Precision: 0.8580
Recall   : 0.9143

Epoch 107/120
Progress: 100.00% | Loss: 0.8000
Loss: 41.8914
Accuracy : 0.9923
Precision: 0.8589
Recall   : 0.9174

Epoch 108/120
Progress: 100.00% | Loss: 0.2199
Loss: 43.0821
Accuracy : 0.9921
Precision: 0.8562
Recall   : 0.9148

Epoch 109/120
Progress: 100.00% | Loss: 0.1970
Loss: 41.2253
Accuracy : 0.9924
Precision: 0.8604
```

```
Recall   : 0.9202
Model Saved!

Epoch 110/120
Progress: 100.00% | Loss: 0.0541
Loss: 40.8804
Accuracy : 0.9924
Precision: 0.8598
Recall   : 0.9200
Model Saved!

Epoch 111/120
Progress: 100.00% | Loss: 0.8125
Loss: 41.7190
Accuracy : 0.9923
Precision: 0.8598
Recall   : 0.9194

Epoch 112/120
Progress: 100.00% | Loss: 0.1031
Loss: 40.5674
Accuracy : 0.9923
Precision: 0.8612
Recall   : 0.9171
Model Saved!

Epoch 113/120
Progress: 100.00% | Loss: 0.0801
Loss: 40.6814
Accuracy : 0.9923
Precision: 0.8579
Recall   : 0.9220

Epoch 114/120
Progress: 100.00% | Loss: 0.0898
Loss: 39.9431
Accuracy : 0.9926
Precision: 0.8655
Recall   : 0.9193
Model Saved!

Epoch 115/120
Progress: 100.00% | Loss: 0.8002
Loss: 42.1142
Accuracy : 0.9924
Precision: 0.8599
Recall   : 0.9198

Epoch 116/120
Progress: 100.00% | Loss: 0.0697
```

```
Loss: 40.1484
Accuracy : 0.9925
Precision: 0.8635
Recall   : 0.9193

Epoch 117/120
Progress: 100.00% | Loss: 0.1174
Loss: 40.1829
Accuracy : 0.9926
Precision: 0.8650
Recall   : 0.9213

Epoch 118/120
Progress: 100.00% | Loss: 0.0883
Loss: 39.8086
Accuracy : 0.9926
Precision: 0.8641
Recall   : 0.9216
Model Saved!

Epoch 119/120
Progress: 100.00% | Loss: 0.3031
Loss: 39.7094
Accuracy : 0.9927
Precision: 0.8652
Recall   : 0.9224
Model Saved!

Epoch 120/120
Progress: 100.00% | Loss: 0.0854
Loss: 40.7114
Accuracy : 0.9925
Precision: 0.8621
Recall   : 0.9211
```

```python
def calculate_all_metrics(pred, target):

    pred = (pred > 0.5).float()

    TP = (pred * target).sum()

    TN = ((1-pred) * (1-target)).sum()

    FP = (pred * (1-target)).sum()

    FN = ((1-pred) * target).sum()

    accuracy = (

        TP + TN
```

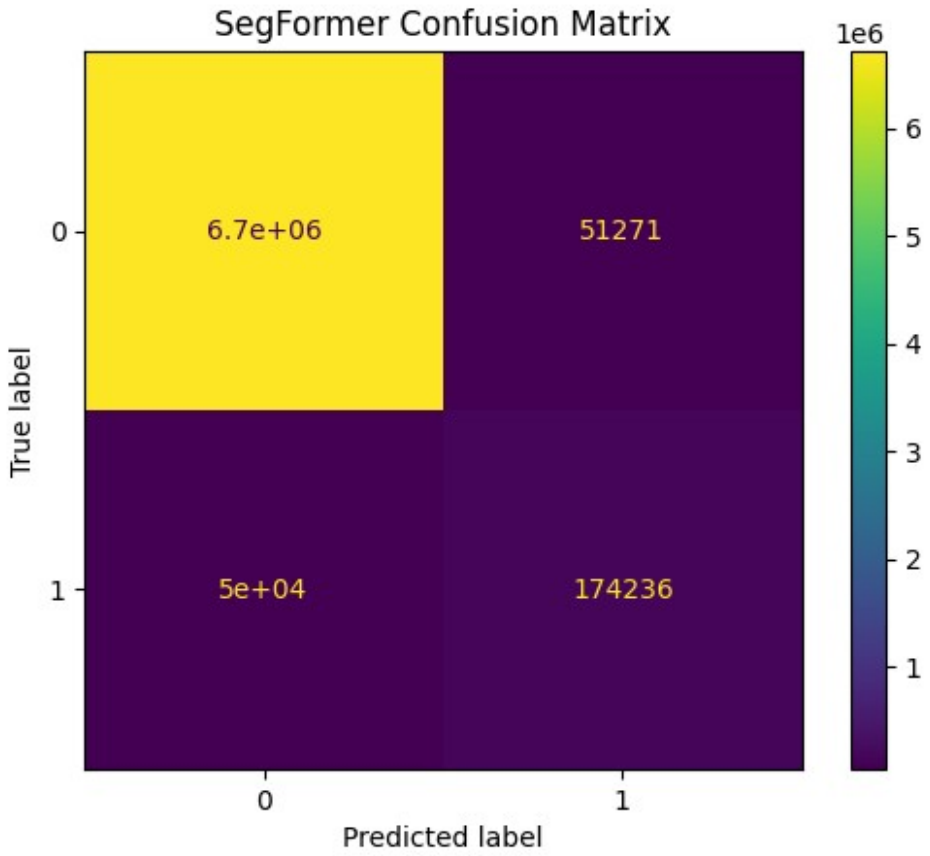
```
    ) / (
        TP + TN + FP + FN + 1e-8
    )
    precision = TP / (
        TP + FP + 1e-8
    )
    recall = TP / (
        TP + FN + 1e-8
    )
    f1 = (
        2 * TP
    ) / (
        2 * TP + FP + FN + 1e-8
    )
    dice = (
        2 * TP
    ) / (
        2 * TP + FP + FN + 1e-8
    )
    iou = TP / (
        TP + FP + FN + 1e-8
    )
    return (
        accuracy.item(),
        precision.item(),
        recall.item(),
```

```
        f1.item(),
        dice.item(),
        iou.item()
    )
model.load_state_dict(
    torch.load(
        "segformer_best.pth"
    )
)
model.eval()
acc = 0
prec = 0
rec = 0
f1_total = 0
dice_total = 0
iou_total = 0
count = 0
all_preds = []
all_targets = []
for images, masks in test_loader:
    images = images.to(device)
    masks = masks.to(device)
    with torch.no_grad():
        pred = model(images)
        pred = torch.sigmoid(pred)
        a, p, r, f1, d, i = calculate_all_metrics(
            pred,
```

```
            masks
        )
        acc += a
        prec += p
        rec += r
        f1_total += f1
        dice_total += d
        iou_total += i
        count += 1
        pred_bin = (
            pred > 0.5
        ).cpu().numpy().flatten()
        target_bin = masks.cpu().numpy().flatten()
        all_preds.extend(pred_bin)
        all_targets.extend(target_bin)
print("\nFINAL RESULTS\n")
print("Accuracy :", acc/count)
print("Precision:", prec/count)
print("Recall   :", rec/count)
print("F1 Score :", f1_total/count)
print("Dice     :", dice_total/count)
print("IoU      :", iou_total/count)
cm = confusion_matrix(
    all_targets,
    all_preds
)
```

```
ConfusionMatrixDisplay(cm).plot()

plt.title(

    "SegFormer Confusion Matrix"

)

plt.show()
```

```
FINAL RESULTS

Accuracy : 0.9855598495765165
Precision: 0.7641394405879758
Recall   : 0.7630306768485091
F1 Score : 0.7522048750384287
Dice     : 0.7522048750384287
IoU      : 0.6148381036790934
```




```
# ================= VALIDATION RESULTS =================

val_accuracy = acc / count
```

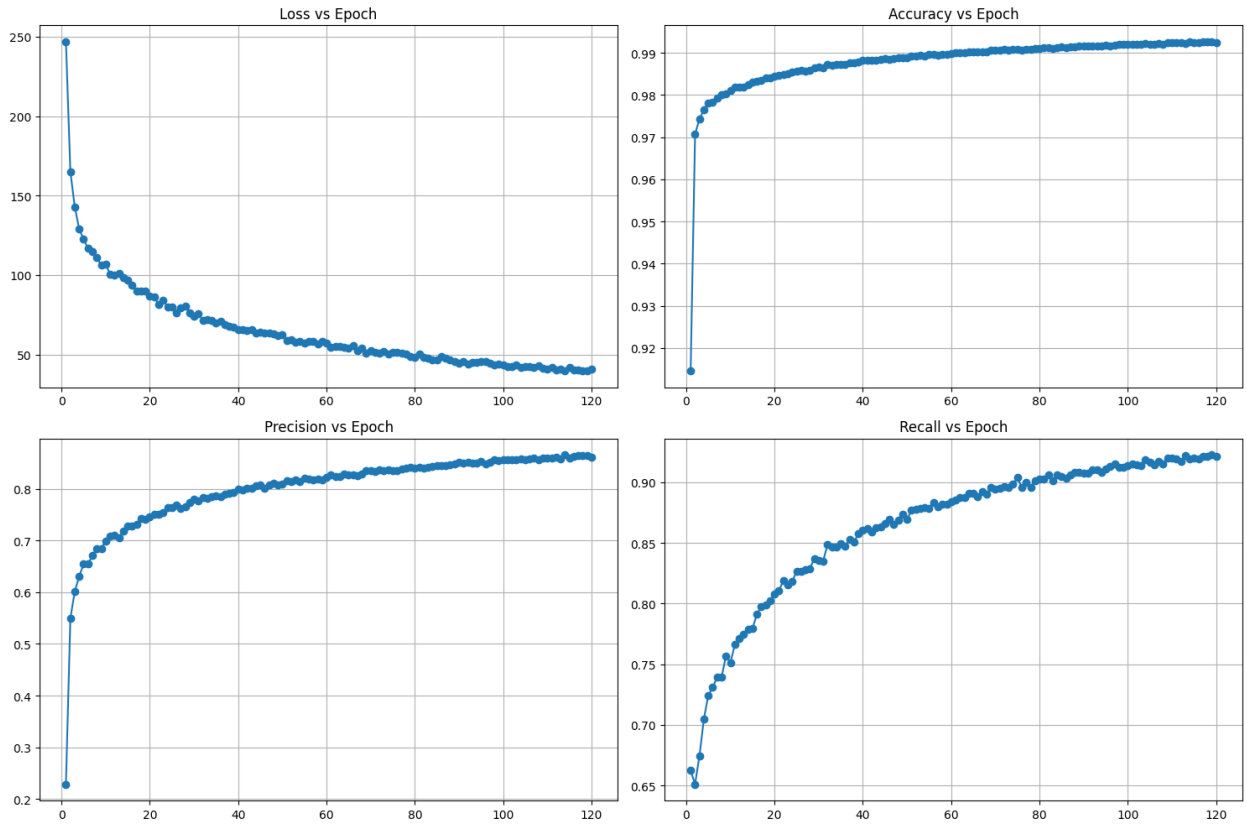
```
val_precision = prec / count
val_recall = rec / count
val_f1 = f1_total / count
val_dice = dice_total / count
val_iou = iou_total / count

print("\nVALIDATION RESULTS\n")

print("Validation Accuracy :", val_accuracy)
print("Validation Precision:", val_precision)
print("Validation Recall   :", val_recall)
print("Validation F1 Score :", val_f1)
print("Validation Dice     :", val_dice)
print("Validation IoU      :", val_iou)
```

```
VALIDATION RESULTS

Validation Accuracy : 0.9855598495765165
Validation Precision: 0.7641394405879758
Validation Recall   : 0.7630306768485091
Validation F1 Score : 0.7522048750384287
Validation Dice     : 0.7522048750384287
Validation IoU      : 0.6148381036790934
```

```
epochs_range = range(

    1,

    len(train_losses)+1

)

plt.figure(figsize=(15,10))

# ================= LOSS =================

plt.subplot(2,2,1)

plt.plot(

    epochs_range,

    train_losses,

    marker='o'

)

plt.title(
```

```
    "Loss vs Epoch"
)
plt.grid()
# ================= ACCURACY =================
plt.subplot(2,2,2)
plt.plot(
    epochs_range,
    train_acc,
    marker='o'
)
plt.title(
    "Accuracy vs Epoch"
)
plt.grid()
# ================= PRECISION =================
plt.subplot(2,2,3)
plt.plot(
    epochs_range,
    train_prec,
    marker='o'
)
plt.title(
    "Precision vs Epoch"
)
plt.grid()
# ================= RECALL =================
```

```
plt.subplot(2,2,4)
plt.plot(
    epochs_range,
    train_rec,
    marker='o'
)
plt.title(
    "Recall vs Epoch"
)
plt.grid()
plt.tight_layout()
plt.show()
```

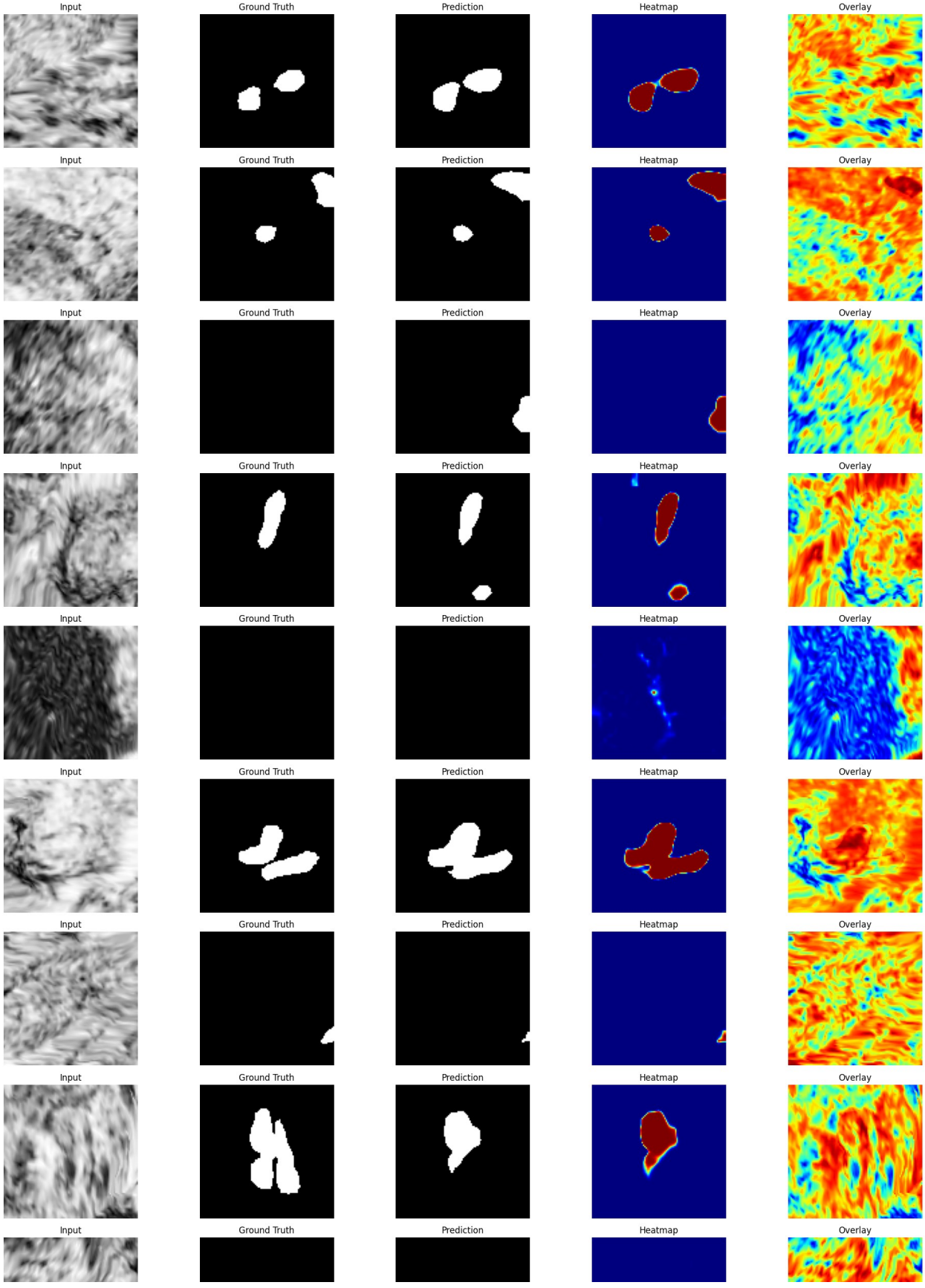


```
model.eval()
images_collected = 0
num_images = 10
all_inputs = []
all_masks = []
all_preds = []
for images, masks in test_loader:
    images = images.to(device)
    with torch.no_grad():
        pred = model(images)
        pred = torch.sigmoid(pred)
    all_inputs.extend(
        images.cpu().numpy()
    )
    all_masks.extend(
        masks.numpy()
    )
    all_preds.extend(
        pred.cpu().numpy()
    )
    images_collected += images.shape[0]
    if images_collected >= num_images:
        break
all_inputs = np.array(all_inputs)
all_masks = np.array(all_masks)
all_preds = np.array(all_preds)
```

```
plt.figure(figsize=(20,30))
for i in range(num_images):
    input_img = all_inputs[i][0]
    true_mask = all_masks[i][0]
    pred_mask = all_preds[i][0]
    binary_pred = (
        pred_mask > 0.5
    ).astype(float)
    heatmap = pred_mask
    overlay = (
        0.6 * input_img
        +
        0.4 * heatmap
    )
    # INPUT
    plt.subplot(
        num_images,
        5,
        i*5 + 1
    )
    plt.title("Input")
    plt.imshow(
        input_img,
        cmap='gray'
    )
    plt.axis('off')
```

```
    # GROUND TRUTH
    plt.subplot(
        num_images,
        5,
        i*5 + 2
    )
    plt.title("Ground Truth")
    plt.imshow(
        true_mask,
        cmap='gray'
    )
    plt.axis('off')
    # PREDICTION
    plt.subplot(
        num_images,
        5,
        i*5 + 3
    )
    plt.title("Prediction")
    plt.imshow(
        binary_pred,
        cmap='gray'
    )
    plt.axis('off')
    # HEATMAP
    plt.subplot(
```

```
        num_images,
        5,
        i*5 + 4
    )
    plt.title("Heatmap")
    plt.imshow(
        heatmap,
        cmap='jet'
    )
    plt.axis('off')
    # OVERLAY
    plt.subplot(
        num_images,
        5,
        i*5 + 5
    )
    plt.title("Overlay")
    plt.imshow(
        overlay,
        cmap='jet'
    )
    plt.axis('off')
plt.tight_layout()
plt.show()
```


Input
Ground Truth
Prediction
Heatmap
Overlay